

[TA] [Kafka] Топики для уведомлений

1. Топики

1.1 payment_notifications

1.1.1 Producer: Payment Service

Триггер: Успешная оплата курса и публикация события `PAYMENT_SUCCESS`.

Требования:

- Producer публикует сообщение в топик при успешной обработке платежа
- В случае неудачи Producer выполняет повторные попытки согласно параметру `retries`
- После исчерпания попыток Producer логирует ошибку с уровнем ERROR

1.1.2 Consumer Group: notification-sender-group

Требования:

- Consumer вычитывает сообщения из топика `payment_notifications`
- При получении сообщения проверяет наличие обязательных полей (`email` , `userId` , `courseName`)
- Вызывает интеграционный метод для отправки уведомлений через Unisender
- При успешной обработке отправляет подтверждение (commit) брокеру
- При ошибке выполняет повторные попытки с экспоненциальной задержкой

1.1.3 Формат сообщения

Назначение сообщения — передать в Notification Service данные для отправки уведомлений (email и SMS) об успешной покупке курса.

Ключ сообщения Kafka: `orderId`. Это обеспечивает последовательную обработку сообщений, относящихся к одному заказу.

Параметр	Расположение	Тип	Обязательность	Описание параметра	Источник данных
<code>eventId</code>	Header	UUID / String	Да	Уникальный идентифи	Генерируется

				котор сообщени я	Notificatio n Service
eventTy pe	Header	String	Да	Тип сообщени я. Постоянн ое значение: Payment Success Event	Задаётся Payment Service
eventVe rsion	Header	String	Да	Версия формата сообщени я. Например , 1.0	Задаётся в конфигур ации или коде Payment Service
occurre dAt	Header	String, date-time ISO 8601	Да	Дата и время формиров ания сообщени я в UTC	Текущее системно е время Payment Service
correla tionId	Header	UUID / String	Да	Идентифи катор исходного пользоват ельского запроса или сквозной идентифи	Заголовок входного HTTP- запроса либо генерируе тся API Gateway

				катор процесса	
<code>source</code>	Header	String	Да	Наименование сервиса — отправителя сообщения. Значение: <code>payment - service</code>	Задаётся Payment Service
<code>userId</code>	Body	String	Да	Идентификатор пользователя, совершившего покупку	Из тела запроса <code>/payments/process → users.id</code>
<code>email</code>	Body	String	Да	Email пользователя для отправки уведомления	Из профиля пользователя → <code>users.email</code>
<code>phone</code>	Body	String	Нет	Номер телефона пользователя для отправки SMS	Из профиля пользователя → <code>users.phone</code>
<code>orderId</code>	Body	String	Да	Уникальный	Из тела запроса

				идентификатор заказа. Используется как ключ сообщения Kafka	<code>/payments/process → orders.id</code>
<code>courseId</code>	Body	String	Да	Идентификатор оплаченного курса	Из тела запроса <code>/payments/process → courses.id</code>
<code>courseName</code>	Body	String	Да	Название оплаченного курса	Из БД курсов <code>→ courses.name</code>
<code>coursePrice</code>	Body	Float	Да	Стоимость оплаченного курса	Из тела запроса <code>/payments/process → orders.total_amount</code>
<code>paymentMethod</code>	Body	String	Да	Способ оплаты (card, sbp)	Из тела запроса <code>/payments/process → payments.payment</code>

					nt_method
payment Id	Body	String	Да	Уникальный идентификатор платежа	Из тела запроса /payments/process → payment.s.id

1.1.4 Пример сообщения

Топик: payment_notifications

Заголовки Kafka:

```

1 {
2   "eventId": "550e8400-e29b-41d4-a716-446655440000",
3   "eventType": "PaymentSuccessEvent",
4   "eventVersion": "1.0",
5   "occurredAt": "2026-08-22T10:30:00Z",
6   "correlationId": "cf99df08-0829-4614-8da3-0e440fd23fe0",
7   "source": "payment-service"
8 }
```

Тело сообщения:

```

1 {
2   "userId": "u_12345",
3   "email": "user@example.com",
4   "phone": "+79991234567",
5   "orderId": "ORD-2026-08-001",
6   "courseId": "course_12345",
7   "courseName": "Основы программирования на Python",
8   "coursePrice": 4990.00,
9   "paymentMethod": "card",
10  "paymentId": "pay_12345"
11 }
```

1.2 notification_status (опционально)

1.2.1 Producer: Notification Service

Триггер: Получение результата отправки уведомления (успех/ошибка).

Назначение: Аудит отправленных уведомлений.

1.2.2 Consumer Group: notification-audit-group

Назначение: Сохранение статусов отправки в БД для истории.

1.2.3 Формат сообщения

Параметр	Расположение	Тип	Обязательность	Описание параметра	Источник данных
<code>eventId</code>	Header	UUID / String	Да	Уникальный идентификатор сообщения	Генерируется Notification Service
<code>eventType</code>	Header	String	Да	Тип сообщения. Постоянное значение: <code>NotificationStatusEvent</code>	Задаётся Notification Service
<code>eventVersion</code>	Header	String	Да	Версия формата сообщения. Например, <code>1.0</code>	Задаётся в конфигурации и Notification Service
<code>occurredAt</code>	Header	String, date-time ISO 8601	Да	Дата и время формирования сообщения в UTC	Текущее системное время Notification Service
<code>source</code>	Header	String	Да	Наименование сервиса — отправителя. Значение: <code>notification-service</code>	Задаётся Notification Service
<code>notificationId</code>	Body	String	Да	Уникальный идентификатор уведомления	Из БД уведомлений →

					notifications.id
userId	Body	String	Да	Идентификатор пользователя	Из исходного сообщения → notifications.user_id
orderId	Body	String	Да	Идентификатор заказа	Из исходного сообщения → notifications.order_id
channel	Body	String	Да	Канал отправки (email, sms)	Определяется в сервисе → notifications.channel
status	Body	String	Да	Статус отправки (SUCCESS, FAILED, PARTIAL_SUCCESS)	Из ответа от Unisender → notifications.status
errorMessage	Body	String	Нет	Сообщение об ошибке (при статусе FAILED)	Из ответа от Unisender → notifications.error_message
timestamp	Body	datetime	Да	Время отправки	Текущее системное время Notification Service →

					notification ons.create d_at
--	--	--	--	--	------------------------------------

1.2.4 Пример сообщения

Топик: notification_status

Заголовки Kafka:

```

1 {
2   "eventId": "550e8400-e29b-41d4-a716-446655440001",
3   "eventType": "NotificationStatusEvent",
4   "eventVersion": "1.0",
5   "occurredAt": "2026-08-22T10:31:00Z",
6   "source": "notification-service"
7 }
```

Тело сообщения:

```

1 {
2   "notificationId": "ntf_67890",
3   "userId": "u_12345",
4   "orderId": "ORD-2026-08-001",
5   "channel": "email",
6   "status": "SUCCESS",
7   "timestamp": "2026-08-22T10:31:00Z"
8 }
```

2. Нефункциональные требования

1. Использовать политику `at-least-once` для обеспечения надежной доставки сообщений. Это гарантирует, что каждое сообщение будет обработано хотя бы один раз, даже в случае сбоев.
2. Все сообщения должны быть сериализованы в формате JSON для унификации и удобства обработки.
3. Время жизни сообщения в топике составляет 7 дней. По истечении этого срока сообщение автоматически удаляется из топика.
4. Для топика `payment_notifications` настроить `replication factor = 3` для обеспечения отказоустойчивости и сохранности данных при сбоях брокера.
5. Для топика `payment_notifications` использовать 3 партии для обеспечения параллельной обработки сообщений.
6. При ошибках отправки уведомлений выполнять до 5 повторных попыток с экспоненциальной задержкой (1с, 2с, 4с, 8с, 16с).
7. Настроить мониторинг задержки обработки сообщений (lag) в consumer group. При превышении порога в 1000 сообщений отправить оповещение администратору.

8. Обработка сообщений должна быть идемпотентной, чтобы избежать дублирования уведомлений при повторной доставке сообщений из Kafka.
9. Все операции публикации и потребления сообщений должны логироваться с указанием времени, топика, partition, offset, ключа и результата обработки.
10. При ошибках публикации сообщений в Kafka Producer выполняет повторные попытки согласно параметру `retries`. После исчерпания попыток логируется ошибка с уровнем ERROR.

3. Взаимодействие сервисов с топиком

- Payment Service → публикует в `payment_notifications` для запуска отправки уведомлений
- Notification Service → вычитывает из `payment_notifications` (Consumer Group: `notification-sender-group`)
- (Опционально) Notification Service → публикует в `notification_status` для аудита
- (Опционально) Audit Service → вычитывает из `notification_status` (Consumer Group: `notification-audit-group`)